

Quản lý Test Case trên GitHub

Từ test case → test run → bug report → pull request → retest

Software Testing

REQUIREMENT → TEST CASE → BUG → FIX → RETEST



Mục tiêu buổi học

Sau buổi học, sinh viên có thể thiết lập một quy trình kiểm thử cơ bản trên GitHub

- 1 Lưu trữ test case có cấu trúc**
Dùng Markdown trong repository hoặc Issue template
- 2 Execute test case có bằng chứng**
Ghi test run, result, tester, ngày chạy
- 3 Tạo bug từ test case fail**
Liên kết Found by Test Case + Related bug
- 4 Theo dõi sửa lỗi bằng GitHub Issues/Projects**
Labels, assignee, milestone, status board
- 5 Retest và traceability**
Requirement ↔ Test Case ↔ Bug ↔ Pull Request

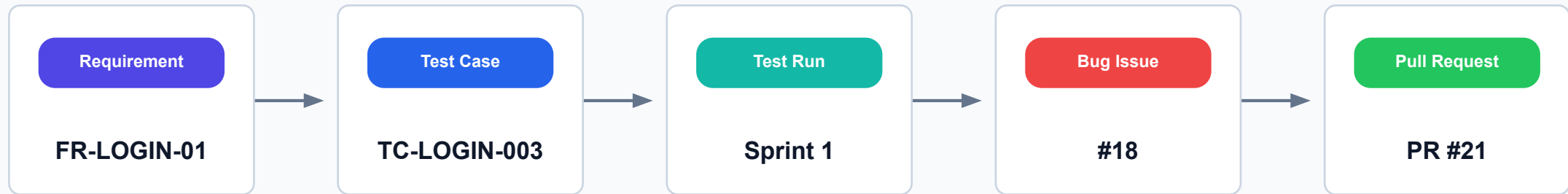
Kết quả cuối cùng

Mỗi lỗi phát hiện đều truy vết được:

Test case nào phát hiện?
Bug issue nào mô tả?
PR nào sửa?
Tester nào retest?

Mô hình tư duy: Test Case khác Bug như thế nào?

Test case là tài sản kiểm thử; bug là kết quả phát hiện khi execute test case



Nguyên tắc:
Một bug phải ghi rõ “Found by Test Case ID”.
Một test case fail phải có “Related bug #...”.

Ba cách quản lý test case trên GitHub

Tùy quy mô dự án, có thể chọn file Markdown, Issues hoặc mô hình Hybrid

1. File Markdown trong repo

```
/tests/test-cases/*.md
```

Khuyến nghị cho lớp học: có version history, review bằng Pull Request, dễ chấm điểm.

2. Mỗi test case là một Issue

```
label: type: test-case
```

Dễ thao tác trên UI, dễ assign và lọc; nhưng dễ rối nếu có nhiều test case.

3. Hybrid

Test case = file; Bug = Issue

Tốt nhất: test case là tài liệu kiểm thử, bug là luồng xử lý lỗi.

Cấu trúc thư mục đề xuất trong repository

Tổ chức để giảng viên và nhóm dễ kiểm tra, review và thống kê

```
project-root/
├── src/
├── tests/
│   ├── test-cases/
│   │   ├── login/
│   │   │   ├── TC-LOGIN-001.md
│   │   │   └── TC-LOGIN-002.md
│   │   ├── register/
│   │   └── checkout/
│   ├── test-runs/
│   │   ├── sprint-1-test-run.md
│   │   └── sprint-2-regression.md
│   ├── test-summary/
│   │   └── traceability-matrix.md
└── .github/ISSUE_TEMPLATE/
```

Vai trò từng thư mục

test-cases/: thiết kế test case chính thức

test-runs/: kết quả thực thi theo sprint hoặc đợt regression

test-summary/: báo cáo tổng hợp và traceability matrix

.github/ISSUE_TEMPLATE/: mẫu Bug Report, Test Task, Test Run

Quy tắc: Không sửa test case trực tiếp trên main.

Tạo branch + Pull Request để review thay đổi test case.

Quy ước đặt mã test case

Mã test case là khóa để liên kết requirement, test run và bug

Cấu trúc đề xuất

TC-[MODULE]-[NUMBER]

TC-LOGIN-001
TC-REGISTER-005
TC-CART-003
TC-CHECKOUT-010

Mã	Ý nghĩa
TC-LOGIN-001	Test case số 1 của Login
TC-REGISTER-005	Test case số 5 của Register
TC-CHECKOUT-010	Test case số 10 của Checkout

Không nên dùng

test1
check-login
case-a
login-success-test-v2-final

Mã test case ổn định giúp traceability không bị đứt khi đổi tiêu đề test case.

Template file Test Case chuẩn

Mỗi test case nên đủ thông tin để người khác execute lại được

```
# TC-LOGIN-001: Đăng nhập thành công
```

```
## Requirement ID  
FR-LOGIN-01
```

```
## Module / Test type / Technique  
Login / Functional / Equivalence Partitioning
```

```
## Preconditions  
- User đã có tài khoản hợp lệ  
- User đang ở trang Login
```

```
## Test data  
| Email | user01@gmail.com |  
| Password | Abc@123456 |
```

```
## Test steps  
1. Mở trang Login  
2. Nhập email và password hợp lệ  
3. Bấm Login
```

```
## Expected result  
Đăng nhập thành công và chuyển về Home.
```

```
## Status / Related bugs  
Not Run / None
```



Requirement ID

Để đo coverage



Preconditions

Tránh lỗi do môi trường/test data



Test data

Rõ dữ liệu hợp lệ/không hợp lệ



Expected result

Căn cứ xác định pass/fail



Related bugs

Liên kết bug khi test fail

Test case tốt không chỉ có steps, mà phải có điều kiện, dữ liệu và expected result rõ ràng.

Test Run: ghi nhận kết quả execute test case

Test case là thiết kế; test run là bằng chứng đã chạy trong một sprint/release

Test Case ID	Module	Tester	Result	Related Bug	Note
TC-LOGIN-001	Login	An	Pass		
TC-LOGIN-002	Login	Bình	Fail	#18	Không validate password
TC-REG-001	Register	Chi	Blocked	#19	Không mở được form
TC-CART-003	Cart	Dũng	Pass		

Trạng thái test run

Pass

Fail

Blocked

Not Run

Khi Result = Fail hoặc Blocked → phải có Related Bug hoặc lý do rõ ràng.

Labels cần có cho quản lý test case và bug

Dùng tiền tố để tạo “nhóm label” để lọc và dễ thống kê

Type

type: test-case

type: test-run

type: bug

type: task

Module

module: login

module: register

module: cart

module: checkout

module: api

Technique

technique: EP

technique: BVA

technique: decision-table

technique: state-transition

Result/Status

result: pass

result: fail

result: blocked

status: ready for retest

Priority/Severity

severity: critical

severity: major

priority: P0

priority: P1

Filter mẫu: is:issue label:"type: bug" label:"status: ready for retest"

GitHub Project cho Test Execution

Project board giúp nhìn toàn bộ tiến độ kiểm thử trong sprint

Not Run

TC-LOGIN

TC-CART

TC-API

Running

TC-LOGIN

TC-CART

TC-API

Passed

TC-LOGIN

TC-CART

TC-API

Failed

TC-LOGIN

TC-CART

TC-API

Blocked

TC-LOGIN

TC-CART

TC-API

Retest

TC-LOGIN

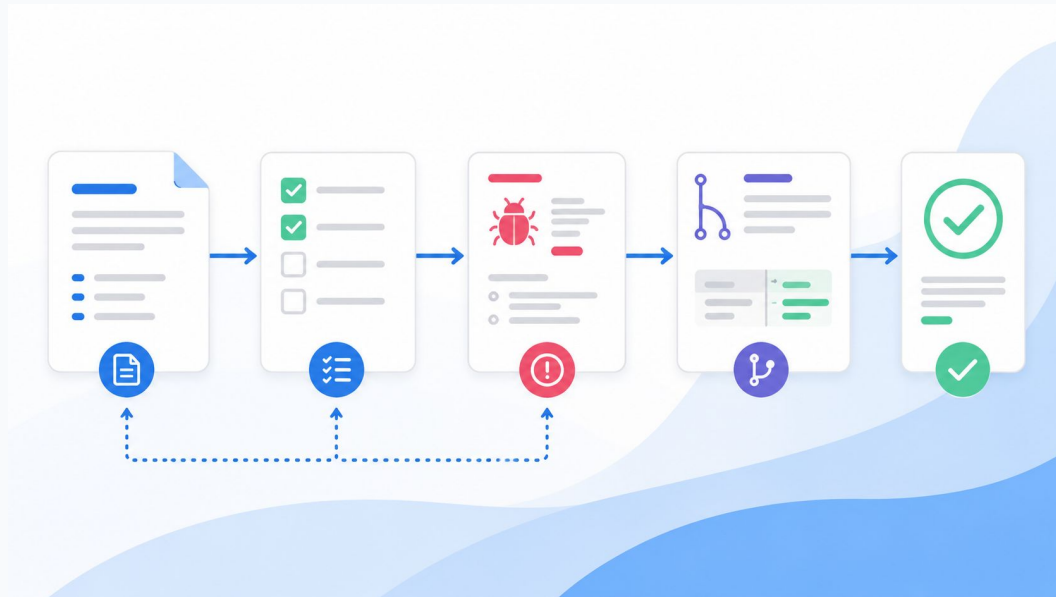
TC-CART

TC-API

Custom field	Ví dụ
Test Case ID	TC-LOGIN-003
Module	Login / Register / Cart
Result	Pass / Fail / Blocked
Related Bug	#18
Tester	Nguyễn Văn A
Execution Date	08/06/2026

Cách kết hợp Test Case và Bug

Một test fail phải sinh ra bug issue có liên kết hai chiều



TC-LOGIN-003 fail

Create Bug Issue #18

PR #21 fixes #18

Retest TC-LOGIN-003

Close #18

Liên kết hai chiều

Trong Bug Issue: ghi Found by Test Case:
TC-LOGIN-003

Trong file Test Run: ghi Result = Fail, Related Bug = #18

Trong file Test Case: bổ sung Related bugs nếu bug nghiêm trọng hoặc tái diễn

Trong Pull Request: ghi Fixes #18 hoặc Related to #18

Không tạo bug chung chung. Bug phải truy ngược được test case nào đã phát hiện.

Template Bug Report khi Test Case fail

Bug là Issue riêng; không chỉ ghi comment trong file test case

Title: [BUG][Login] Hệ thống cho phép đăng nhập với password sai

Found by Test Case
TC-LOGIN-003

Requirement liên quan
FR-LOGIN-02

Severity / Priority
Major / P1

Environment
Browser, OS, URL, build/commit

Steps to reproduce
1. Mở trang Login
2. Nhập email hợp lệ
3. Nhập password sai
4. Bấm Login

Expected result
Không cho đăng nhập và hiển thị lỗi.

Actual result
Hệ thống vẫn đăng nhập thành công.

Evidence
Screenshot / video / console log

Labels nên gắn

type: bug

module: login

severity: major

priority: P1

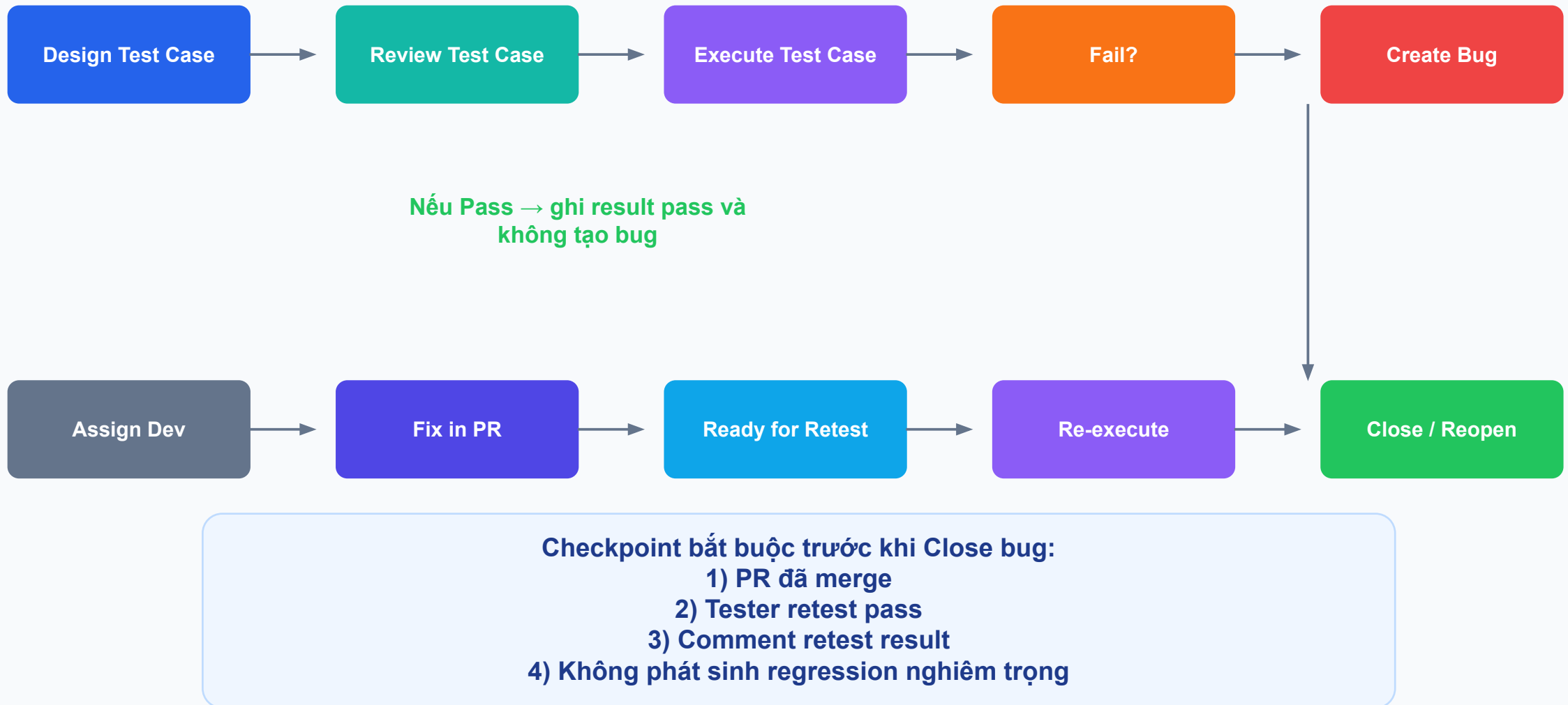
status: new

found-by: test-case

**Nếu thiếu Expected/Actual/Evidence,
developer rất khó sửa và tester khó
retest.**

Workflow chuẩn từ thiết kế test case đến đóng bug

Có checkpoint rõ ràng cho tester, developer và test lead



Traceability Matrix: truy vết Requirement - Test Case - Bug

Bảng này giúp chứng minh coverage và không bỏ sót lỗi

Requirement	Test Case	Result	Bug Issue	Status
FR-LOGIN-01	TC-LOGIN-001	Pass		Done
FR-LOGIN-02	TC-LOGIN-003	Fail	#18	Ready for Retest
FR-REGISTER-01	TC-REGISTER-001	Pass		Done
FR-REGISTER-02	TC-REGISTER-005	Fail	#24	Open
FR-CART-01	TC-CART-001	Blocked	#27	Blocked

Coverage

Requirement nào đã test?

Defect traceability

Bug nào thuộc requirement nào?

Regression

Test case nào cần chạy lại?

Kết hợp với Automated Test và GitHub Actions

Khi có automation, test report có thể lưu dưới dạng artifact và liên kết về bug

```
name: Run Tests

on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm install
      - run: npm test
      - uses: actions/upload-artifact@v4
        with:
          name: test-report
          path: reports/
```

Khi test tự động fail

Kiểm tra log/test report trong GitHub Actions

Xác định test case hoặc test script bị fail

Tạo Bug Issue nếu lỗi do hệ thống, không phải lỗi test script

Trong bug ghi: Found by: GitHub Actions + workflow run + test case/script

Sau khi fix, workflow phải pass trước khi close bug

**Automation không thay thế test case;
automation là cách execute một phần test case
nhANH và lặp lại.**